第7章 函数

7.1 学习目标

- 理解函数的概念与模块化编程思想
- 掌握函数的定义、声明和调用
- 理解实参与形参的传递机制
- 掌握递归函数
- 理解变量的作用域和生命周期
- 使用多文件编程

7.2 为什么需要函数

```
// 没有函数：代码重复，难以维护
int main()
{
    // 计算 5!
    int fact1 = 1;
    for (int i = 1; i <= 5; i++) fact1 *= i;

    // 计算 8!
    int fact2 = 1;
    for (int i = 1; i <= 8; i++) fact2 *= i;

    // 计算 10!
    int fact3 = 1;
    for (int i = 1; i <= 10; i++) fact3 *= i;
}

// 有函数：代码复用，简洁清晰
int factorial(int n)
{
    int result = 1;
    for (int i = 1; i <= n; i++)
        result *= i;
    return result;
}

int main()
{
    int r1 = factorial(5);    // 调用
    int r2 = factorial(8);
    int r3 = factorial(10);
}
```

> 函数 = 完成特定功能的独立代码块，实现"一次编写，多次调用"。

7.3 函数的定义

7.3.1 定义格式

```
返回类型 函数名(形参列表)
{
    函数体;
    return 返回值;    // 如果返回类型是void，可省略
}
```

```
// 示例1：有参数有返回值
int add(int a, int b)
{
    return a + b;
}

// 示例2：有参数无返回值
void printStars(int n)
{
    for (int i = 0; i < n; i++)
        printf("*");
    printf("\n");
    // 没有return，函数执行完自动返回
}

// 示例3：无参数有返回值
int getNumber()
{
    int num;
    printf("请输入一个数: ");
    scanf("%d", &num);
    return num;
}

// 示例4：无参数无返回值
void printHeader()
{
    printf("====================\n");
    printf("    学生管理系统\n");
    printf("====================\n");
}
```

7.4 函数的声明与调用

7.4.1 函数声明（原型）

```
#include <stdio.h>
```

```c
// 函数声明（告诉编译器有这个函数）：可以省略形参名
int max(int a, int b);   // 或 int max(int, int);

int main()
{
    int result = max(10, 20);   // 调用
    printf("较大值: %d\n", result);
    return 0;
}

// 函数定义
int max(int a, int b)
{
    return (a > b) ? a : b;
}
```

> **规则**：函数定义在调用之后时，必须先声明。定义在调用之前则不需要。

7.4.2 函数调用过程

```c
#include <stdio.h>

int square(int n)       // n 是形参
{
    return n * n;
}

int main()
{
    int x = 5;
    int result = square(x);  // x 是实参
    printf("%d? = %d\n", x, result);
    return 0;
}
```

```
调用过程:
1. 实参 x 的值(5) 复制给 形参 n
2. 跳转到 square 函数执行
3. 计算 n*n = 25
4. return 将 25 返回给调用处
5. result 接收返回值
6. 继续执行 main 的后续代码
```

7.5 参数传递机制

7.5.1 值传递（重点）

> **C语言参数传递是"值传递"**：将实参的值复制一份给形参，函数内修改形参不影响实参。

```c
#include <stdio.h>
```

```c
void swap(int a, int b)    // 值传递，无法交换实参的值
{
    int temp = a;
    a = b;
    b = temp;
    printf("swap内: a=%d, b=%d\n", a, b);
}

int main()
{
    int x = 10, y = 20;
    swap(x, y);
    printf("main内: x=%d, y=%d\n", x, y);  // x=10, y=20 未改变!
    return 0;
}
```

```
内存示意:
main:   x=10    y=20
          ↓复制    ↓复制
swap:   a=10    b=20    → 交换后 → a=20  b=10
         ↑ 不影响 ↑
main:   x=10    y=20    （不变）
```

7.5.2 通过指针修改实参（下章详讲）

```c
// 用指针实现真正的交换（第8章详解）
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main()
{
    int x = 10, y = 20;
    swap(&x, &y);
    printf("x=%d, y=%d\n", x, y);  // x=20, y=10 ?
    return 0;
}
```

7.6 数组作为函数参数

7.6.1 一维数组传参

```c
// 数组作为参数时，实际传递的是数组首地址（指针）
// 以下三种写法等价
float average(int arr[], int n);       // 最常用
```

```
float average(int arr[10], int n);     // 长度忽略
float average(int *arr, int n);        // 指针写法

float average(int arr[], int n)
{
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += arr[i];
    return (float)sum / n;
}

int main()
{
    int scores[] = {85, 92, 78, 90, 88};
    int n = sizeof(scores) / sizeof(scores[0]);
    printf("平均分: %.2f\n", average(scores, n));
    return 0;
}
```

> 数组传参时，**实际传递的是首地址**，函数内可以修改数组元素的值（因为操作的是同一块内存）。

```
// 函数内修改数组
void doubleArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        arr[i] *= 2;           // 会直接影响原数组!
}

int main()
{
    int a[] = {1, 2, 3, 4, 5};
    doubleArray(a, 5);
    for (int i = 0; i < 5; i++)
        printf("%d ", a[i]);  // 输出: 2 4 6 8 10
    return 0;
}
```

7.7 递归函数

7.7.1 什么是递归

> 递归：函数直接或间接地调用自身。

两个必要条件：

1. **递归出口**（终止条件）
2. **递归表达式**（将大问题分解为小问题）

7.7.2 经典案例：阶乘

```
// n! = n * (n-1)!
```

```
// 出口: 0! = 1
int factorial(int n)
{
    if (n <= 1)
        return 1;                  // 递归出口
    else
        return n * factorial(n - 1);  // 递归调用
}

// factorial(4) 的执行过程:
// factorial(4) = 4 * factorial(3)
//                      = 3 * factorial(2)
//                           = 2 * factorial(1)
//                                = 1
// 回溯: 1 → 2 → 6 → 24
```

7.7.3 案例：汉诺塔

```
#include <stdio.h>

// 将 n 个盘子从 A 柱移到 C 柱（B 柱作为辅助）
void hanoi(int n, char A, char B, char C)
{
    if (n == 1)
    {
        printf("%c -> %c\n", A, C);     // 1个盘子直接移动
    }
    else
    {
        hanoi(n - 1, A, C, B);          // 将n-1个从A移到B
        printf("%c -> %c\n", A, C);     // 将最大的从A移到C
        hanoi(n - 1, B, A, C);          // 将n-1个从B移到C
    }
}

int main()
{
    int n;
    printf("请输入盘子数: ");
    scanf("%d", &n);
    hanoi(n, 'A', 'B', 'C');
    return 0;
}
```

7.7.4 递归的优缺点

优点	缺点
代码简洁，思路清晰	效率较低（函数调用开销）
适合解决分治问题	可能栈溢出（递归太深）

优点	缺点
如树的遍历、全排列	占用额外栈空间

7.8 变量的作用域和生命周期

7.8.1 局部变量

```
#include <stdio.h>

void func()
{
    int x = 10;           // 局部变量：仅在func内有效
    printf("func内: %d\n", x);
}
// x 在此处已被销毁

int main()
{
    int x = 5;            // 同名局部变量，互不影响
    func();
    printf("main内: %d\n", x);  // 输出 5
    return 0;
}
```

7.8.2 全局变量

```
#include <stdio.h>

int count = 0;     // 全局变量（在所有函数外定义）

void increment()
{
    count++;        // 直接访问全局变量
}

int main()
{
    increment();
    increment();
    printf("count = %d\n", count);  // 2
    return 0;
}
```

> 全局变量从定义处开始到文件结束都有效。**谨慎使用**，过多全局变量会降低程序可维护性。

7.8.3 static 变量

```
#include <stdio.h>
```

```c
void counter()
{
    static int count = 0;   // 静态局部变量：只初始化一次，值在函数调用间保持
    count++;
    printf("调用第 %d 次\n", count);
}

int main()
{
    for (int i = 0; i < 3; i++)
        counter();
    return 0;
}
/*
输出:
调用第 1 次
调用第 2 次
调用第 3 次
*/
```

7.9 多文件编程

7.9.1 文件结构

```
项目/
├── main.c        # 主程序
├── math_ops.c    # 数学函数实现
└── math_ops.h    # 数学函数声明（头文件）
```

math_ops.h（头文件）

```c
#ifndef MATH_OPS_H          // 防止重复包含
#define MATH_OPS_H

// 函数声明
int add(int a, int b);
int subtract(int a, int b);
int multiply(int a, int b);
double divide(int a, int b);

#endif
```

math_ops.c（实现文件）

```c
#include "math_ops.h"

int add(int a, int b)
{
```

```
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

int multiply(int a, int b)
{
    return a * b;
}

double divide(int a, int b)
{
    if (b == 0) return 0;
    return (double)a / b;
}
```

main.c（主文件）

```
#include <stdio.h>
#include "math_ops.h"     // 引入自定义头文件

int main()
{
    int a = 10, b = 5;
    printf("%d + %d = %d\n", a, b, add(a, b));
    printf("%d - %d = %d\n", a, b, subtract(a, b));
    printf("%d * %d = %d\n", a, b, multiply(a, b));
    printf("%d / %d = %.1f\n", a, b, divide(a, b));
    return 0;
}
```

编译多文件

```
# 方法1：一起编译
gcc main.c math_ops.c -o program

# 方法2：分别编译再链接
gcc -c main.c          # 生成 main.o
gcc -c math_ops.c      # 生成 math_ops.o
gcc main.o math_ops.o -o program
```

7.10 综合案例：小型计算器模块

```
// calc.h
#ifndef CALC_H
#define CALC_H
```

```c
// 打印菜单
void printMenu();

// 四则运算
double add(double a, double b);
double sub(double a, double b);
double mul(double a, double b);
double div(double a, double b);

// 幂运算
double power(double base, int exp);

// 阶乘
long long factorial(int n);

// 判断素数
int isPrime(int n);

#endif
```

```c
// calc.c
#include "calc.h"
#include <stdio.h>

void printMenu()
{
    printf("\n====== 计算器菜单 ======\n");
    printf("1. 加法\n2. 减法\n3. 乘法\n4. 除法\n");
    printf("5. 幂运算\n6. 阶乘\n7. 素数判断\n");
    printf("0. 退出\n");
    printf("请选择: ");
}

double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }
double div(double a, double b)
{
    if (b == 0) { printf("除数不能为零!\n"); return 0; }
    return a / b;
}

double power(double base, int exp)
{
    double result = 1;
    for (int i = 0; i < exp; i++)
        result *= base;
    return result;
}

long long factorial(int n)
```

```
{
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

int isPrime(int n)
{
    if (n <= 1) return 0;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1;
}
```

7.11 本章小结

知识点	要点
函数定义	`返回类型 函数名(参数列表) { 函数体 }`
函数声明	`返回类型 函数名(参数列表);`
值传递	实参值复制给形参，函数内修改不影响实参
数组传参	传递首地址，函数内可修改数组元素
递归	函数调用自身，必须有终止条件
局部变量	函数内部定义的变量，函数结束后销毁
全局变量	所有函数外定义，整个文件可见
static	静态变量只初始化一次，值在调用间保持
头文件	`.h` 文件放声明，`#ifndef` 防重复包含

7.12 课后作业

基础练习

1. 选择题

(1) 以下函数定义的写法正确的是（　）：

A. `int add(a, b) { return a+b; }`　　B. `add(int a, int b) { return a+b; }`

C. `int add(int a, int b) { return a+b; }`　　D. `int add(int a, b) { return a+b; }`

(2) 关于递归函数，以下说法正确的是（　）：

A. 递归函数不能有返回值　　B. 递归函数必须有终止条件

C. 递归一定比循环快　　D. 递归函数不能调用其他函数

2. 程序填空

```c
// 判断是否为闰年的函数
int isLeapYear(int year)
{
    return (year % 4 == 0 && year % 100 != 0) ___ (year % 400 == 0);
}
```

编程练习

3. 求最大公约数函数
编写函数 `int gcd(int a, int b)` 用辗转相除法求最大公约数，并编写 `main` 函数测试。

4. 哥德巴赫猜想验证
编写函数 `int isPrime(int n)` 判断素数，然后在 main 中验证：任何大于2的偶数都可以表示成两个素数之和。（输入一个偶数，输出符合条件的两个素数）

5. 递归求斐波那契数列
用递归函数实现 `int fib(int n)`，返回第 n 项斐波那契数列的值。并思考为什么递归求斐波那契效率低。

6. 字符统计
编写函数 `void countChars(char str[])`，统计字符串中大写字母、小写字母、数字、空格的数量。

7. 矩阵运算（多文件）
创建 `matrix.h` 和 `matrix.c`，实现矩阵加法、矩阵乘法函数。在 `main.c` 中调用测试。

> **学习建议**：函数是程序模块化的基础。写代码时先思考"需要哪些功能"，每个功能写一个函数，main函数只负责调度。